Practice 1 Solutions

1. Initiate an *i*SQL*Plus session using the user ID and password provided by the instructor.
2. *i*SQL*Plus commands access the database.
   False
3. The following `SELECT` statement executes successfully:
   True

```
SELECT last_name, job_id, salary AS Sal
FROM   employees;
```

4. The following `SELECT` statement executes successfully:
   True

```
SELECT *
FROM   job_grades;
```

5. There are four coding errors in this statement. Can you identify them?

```
SELECT    employee_id, last_name
sal x 12  ANNUAL SALARY
FROM      employees;
```

   - **The `EMPLOYEES` table does not contain a column called `sal`. The column is called `SALARY`.**
   - **The multiplication operator is *, not x, as shown in line 2.**
   - **The `ANNUAL SALARY` alias cannot include spaces. The alias should read `ANNUAL_SALARY` or be enclosed in double quotation marks.**
   - **A comma is missing after the column, `LAST_NAME`.**

6. Show the structure of the `DEPARTMENTS` table. Select all data from the `DEPARTMENTS` table.

```
DESCRIBE departments

SELECT *
FROM   departments;
```

7. Show the structure of the `EMPLOYEES` table. Create a query to display the last name, job code, hire date, and employee number for each employee, with employee number appearing first. Save your SQL statement to a file named `lab1_7.sql`.

```
DESCRIBE employees

SELECT employee_id, last_name, job_id, hire_date
FROM   employees;
```

Practice 1 Solutions (continued)

8. Run your query in the file `lab1_7.sql`.

```
SELECT employee_id, last_name, job_id, hire_date
FROM   employees;
```

9. Create a query to display unique job codes from the `EMPLOYEES` table.

```
SELECT DISTINCT job_id
FROM   employees;
```

If you have time, complete the following exercises:

10. Copy the statement from `lab1_7.sql` into the *i*SQL*Plus Edit window. Name the column headings `Emp #`, `Employee`, `Job`, and `Hire Date`, respectively. Run your query again.

```
SELECT employee_id "Emp #", last_name "Employee",
       job_id "Job", hire_date "Hire Date"
FROM   employees;
```

11. Display the last name concatenated with the job ID, separated by a comma and space, and name the column `Employee and Title`.

```
SELECT last_name||', '||job_id "Employee and Title"
FROM   employees;
```

If you want an extra challenge, complete the following exercise:

12. Create a query to display all the data from the `EMPLOYEES` table. Separate each column by a comma. Name the column `THE_OUTPUT`.

```
SELECT employee_id || ',' || first_name || ',' || last_name
      || ',' || email || ',' || phone_number || ','|| job_id
      || ',' || manager_id || ',' || hire_date || ',' ||
      salary || ',' || commission_pct || ',' || department_id
      THE_OUTPUT
FROM   employees;
```

Practice 2 Solutions

1. Create a query to display the last name and salary of employees earning more than $12,000. Place your SQL statement in a text file named `lab2_1.sql`. Run your query.

```
SELECT  last_name, salary
FROM    employees
WHERE   salary > 12000;
```

2. Create a query to display the employee last name and department number for employee number 176.

```
SELECT  last_name, department_id
FROM    employees
WHERE   employee_id = 176;
```

3. Modify `lab2_1.sql` to display the last name and salary for all employees whose salary is not in the range of $5,000 and $12,000. Place your SQL statement in a text file named `lab2_3.sql`.

```
SELECT  last_name, salary
FROM    employees
WHERE   salary NOT BETWEEN 5000 AND 12000;
```

4. Display the employee last name, job ID, and start date of employees hired between February 20, 1998, and May 1, 1998. Order the query in ascending order by start date.

```
SELECT   last_name, job_id, hire_date
FROM     employees
WHERE    hire_date BETWEEN '20-Feb-1998' AND '01-May-1998'
ORDER BY hire_date;
```

Practice 2 Solutions (continued)

5. Display the last name and department number of all employees in departments 20 and 50 in alphabetical order by name.

```
SELECT   last_name, department_id
FROM     employees
WHERE    department_id IN (20, 50)
ORDER BY last_name;
```

6. Modify `lab2_3.sql` to list the last name and salary of employees who earn between $5,000 and $12,000, and are in department 20 or 50. Label the columns `Employee` and `Monthly Salary`, respectively. Resave `lab2_3.sql` as `lab2_6.sql`. Run the statement in `lab2_6.sql`.

```
SELECT   last_name "Employee", salary "Monthly Salary"
FROM     employees
WHERE    salary  BETWEEN 5000 AND 12000
AND      department_id IN (20, 50);
```

7. Display the last name and hire date of every employee who was hired in 1994.

```
SELECT   last_name, hire_date
FROM     employees
WHERE    hire_date LIKE '%94';
```

8. Display the last name and job title of all employees who do not have a manager.

```
SELECT   last_name, job_id
FROM     employees
WHERE    manager_id IS NULL;
```

9. Display the last name, salary, and commission for all employees who earn commissions. Sort data in descending order of salary and commissions.

```
SELECT   last_name, salary, commission_pct
FROM     employees
WHERE    commission_pct IS NOT NULL
ORDER BY salary DESC, commission_pct DESC;
```

Practice 2 Solutions (continued)

If you have time, complete the following exercises.

10. Display the last names of all employees where the third letter of the name is an *a*.

```
SELECT   last_name
FROM     employees
WHERE    last_name LIKE '__a%';
```

11. Display the last name of all employees who have an *a* and an *e* in their last name.

```
SELECT   last_name
FROM     employees
WHERE    last_name LIKE '%a%'
  AND    last_name LIKE '%e%';
```

If you want an extra challenge, complete the following exercises:

12. Display the last name, job, and salary for all employees whose job is sales representative or stock clerk and whose salary is not equal to $2,500, $3,500, or $7,000.

```
SELECT   last_name, job_id, salary
FROM     employees
WHERE    job_id IN ('SA_REP', 'ST_CLERK')
  AND    salary NOT IN (2500, 3500, 7000);
```

13. Modify `lab2_6.sql` to display the last name, salary, and commission for all employees whose commission amount is 20%. Resave `lab2_6.sql` as `lab2_13.sql`. Rerun the statement in `lab2_13.sql`.

```
SELECT   last_name "Employee", salary "Monthly Salary",
         commission_pct
FROM     employees
WHERE    commission_pct = .20;
```

Practice 3 Solutions

1. Write a query to display the current date. Label the column `Date`.

```
SELECT   sysdate "Date"
FROM     dual;
```

2. For each employee, display the employee number, last_name, salary, and salary increased by 15% and expressed as a whole number. Label the column `New Salary`. Place your SQL statement in a text file named `lab3_2.sql`.

```
SELECT   employee_id, last_name, salary,
         ROUND(salary * 1.15, 0) "New Salary"
FROM     employees;
```

3. Run your query in the file `lab3_2.sql`.

```
SELECT  employee_id, last_name, salary,
        ROUND(salary * 1.15, 0) "New Salary"
FROM    employees;
```

4. Modify your query `lab3_2.sql` to add a column that subtracts the old salary from the new salary. Label the column `Increase`. Save the contents of the file as `lab3_4.sql`. Run the revised query.

```
SELECT   employee_id, last_name, salary,
  ROUND(salary * 1.15, 0) "New Salary",
  ROUND(salary * 1.15, 0) - salary "Increase"
FROM     employees;
```

5. Write a query that displays the employee's last names with the first letter capitalized and all other letters lowercase and the length of the name for all employees whose name starts with *J*, *A*, or *M*. Give each column an appropriate label. Sort the results by the employees' last names.

```
SELECT    INITCAP(last_name) "Name",
          LENGTH(last_name) "Length"
FROM      employees
WHERE     last_name LIKE 'J%'
OR        last_name LIKE 'M%'
OR        last_name LIKE 'A%'
ORDER BY last_name;
```

Practice 3 Solutions (continued)

6. For each employee, display the employee's last name, and calculate the number of months between today and the date the employee was hired. Label the column MONTHS_WORKED. Order your results by the number of months employed. Round the number of months up to the closest whole number.

   Note: Your results will differ.

```
SELECT   last_name, ROUND(MONTHS_BETWEEN
               (SYSDATE, hire_date)) MONTHS_WORKED
FROM     employees
ORDER BY MONTHS_BETWEEN(SYSDATE, hire_date);
```

7. Write a query that produces the following for each employee: <employee last name> earns <salary> monthly but wants <3 times salary>. Label the column Dream Salaries.

```
SELECT   last_name || ' earns '
         || TO_CHAR(salary, 'fm$99,999.00')
         || ' monthly but wants '
         || TO_CHAR(salary * 3, 'fm$99,999.00')
         || '.' "Dream Salaries"
FROM     employees;
```

If you have time, complete the following exercises:

8. Create a query to display the last name and salary for all employees. Format the salary to be 15 characters long, left-padded with $. Label the column SALARY.

```
SELECT   last_name,
         LPAD(salary, 15, '$') SALARY
FROM     employees;
```

9. Display each employee's last name, hire date, and salary review date, which is the first Monday after six months of service. Label the column REVIEW. Format the dates to appear in the format similar to "Monday, the Thirty-First of July, 2000."

```
SELECT   last_name, hire_date,
  TO_CHAR(NEXT_DAY(ADD_MONTHS(hire_date, 6),'MONDAY'),
         'fmDay, "the" Ddspth "of" Month, YYYY') REVIEW
FROM     employees;
```

10. Display the last name, hire date, and day of the week on which the employee started. Label the column DAY. Order the results by the day of the week starting with Monday.

```
SELECT   last_name, hire_date,
         TO_CHAR(hire_date, 'DAY') DAY
FROM     employees
ORDER BY TO_CHAR(hire_date - 1, 'd');
```

Practice 3 Solutions (continued)

If you want an extra challenge, complete the following exercises:

11. Create a query that displays the employees' last names and commission amounts. If an employee does not earn commission, put "No Commission." Label the column COMM.

```
SELECT   last_name,
         NVL(TO_CHAR(commission_pct), 'No Commission') COMM
FROM     employees;
```

12. Create a query that displays the employees' last names and indicates the amounts of their annual salaries with asterisks. Each asterisk signifies a thousand dollars. Sort the data in descending order of salary. Label the column EMPLOYEES_AND_THEIR_SALARIES.

```
SELECT   rpad(last_name, 8)||' '|| rpad(' ', salary/1000+1, '*')
         EMPLOYEES_AND_THEIR_SALARIES
FROM     employees
ORDER BY salary DESC;
```

13. Using the DECODE function, write a query that displays the grade of all employees based on the value of the column JOB_ID, as per the following data:

JOB	*GRADE*
AD_PRES	A
ST_MAN	B
IT_PROG	C
SA_REP	D
ST_CLERK	E
None of the above	0

```
SELECT job_id, decode (job_id,
                       'ST_CLERK',  'E',
                       'SA_REP',    'D',
                       'IT_PROG',   'C',
                       'ST_MAN',    'B',
                       'AD_PRES',   'A',
                                    '0')GRADE
FROM employees;
```

Practice 3 Solutions (continued)

14. Rewrite the statement in the preceding question using the `CASE` syntax.

```
SELECT job_id, CASE job_id
       WHEN 'ST_CLERK' THEN 'E'
       WHEN 'SA_REP'   THEN 'D'
       WHEN 'IT_PROG'  THEN 'C'
       WHEN 'ST_MAN'   THEN 'B'
       WHEN 'AD_PRES'  THEN 'A'
       ELSE '0'  END  GRADE
FROM employees;
```

Practice 4 Solutions

1. Write a query to display the last name, department number, and department name for all employees.

```
SELECT e.last_name, e.department_id, d.department_name
FROM employees e, departments d
WHERE e.department_id = d.department_id;
```

2. Create a unique listing of all jobs that are in department 30. Include the location of department 90 in the output.

```
SELECT DISTINCT job_id, location_id
FROM employees, departments
WHERE employees.department_id = departments.department_id
AND employees.department_id = 80;
```

3. Write a query to display the employee last name, department name, location ID, and city of all employees who earn a commission.

```
SELECT e.last_name, d.department_name, d.location_id, l.city
FROM employees e, departments d, locations l
WHERE e.department_id = d.department_id
AND
d.location_id = l.location_id
AND e.commission_pct IS NOT NULL;
```

4. Display the employee last name and department name for all employees who have an *a* (lowercase) in their last names. Place your SQL statement in a text file named `lab4_4.sql`.

```
SELECT last_name, department_name
FROM employees, departments
WHERE employees.department_id = departments.department_id
AND last_name LIKE '%a%';
```

5. Write a query to display the last name, job, department number, and department name for all employees who work in Toronto.

```
SELECT e.last_name, e.job_id, e.department_id,
      d.department_name
FROM employees e JOIN departments d
ON (e.department_id = d.department_id)
JOIN locations l
ON (d.location_id = l.location_id)
WHERE LOWER(l.city) = 'toronto';
```

6. Display the employee last name and employee number along with their manager's last name and manager number. Label the columns `Employee`, `Emp#`, `Manager`, and `Mgr#`, respectively. Place your SQL statement in a text file named `lab4_6.sql`.

```
SELECT w.last_name "Employee", w.employee_id "EMP#",
       m.last_name "Manager", m.employee_id  "Mgr#"
FROM employees w join employees m
ON (w.manager_id = m.employee_id);
```

Practice 4 Solutions (continued)

7. Modify `lab4_6.sql` to display all employees including King, who has no manager. Place your SQL statement in a text file named `lab4_7.sql`. Run the query in `lab4_7.sql`

```
SELECT w.last_name "Employee", w.employee_id "EMP#",
       m.last_name "Manager", m.employee_id  "Mgr#"
FROM employees w
LEFT OUTER JOIN employees m
ON (w.manager_id = m.employee_id);
```

If you have time, complete the following exercises.

8. Create a query that displays employee last names, department numbers, and all the employees who work in the same department as a given employee. Give each column an appropriate label.

```
SELECT e.department_id department, e.last_name employee,
       c.last_name colleague
FROM   employees e JOIN employees c
ON     (e.department_id = c.department_id)
WHERE   e.employee_id <> c.employee_id
ORDER BY e.department_id, e.last_name, c.last_name;
```

9. Show the structure of the `JOB_GRADES` table. Create a query that displays the name, job, department name, salary, and grade for all employees.

```
DESC JOB_GRADES
SELECT e.last_name, e.job_id, d.department_name,
       e.salary, j.grade_level
FROM   employees e, departments d, job_grades j
WHERE  e.department_id = d.department_id
AND    e.salary BETWEEN j.lowest_sal AND j.highest_sal;
-- OR
SELECT e.last_name, e.job_id, d.department_name,
       e.salary, j.grade_level
FROM   employees e JOIN departments d
ON     (e.department_id = d.department_id)
JOIN   job_grades j
ON    (e.salary BETWEEN j.lowest_sal AND j.highest_sal);
```

Practice 4 Solutions (continued)

If you want an extra challenge, complete the following exercises:

10. Create a query to display the name and hire date of any employee hired after employee Davies.

```
SELECT e.last_name, e.hire_date
FROM   employees e, employees davies
WHERE  davies.last_name = 'Davies'
AND    davies.hire_date < e.hire_date
-- OR
SELECT e.last_name, e.hire_date
FROM   employees e JOIN employees davies
ON     (davies.last_name = 'Davies')
WHERE  davies.hire_date < e.hire_date;
```

11. Display the names and hire dates for all employees who were hired before their managers, along with their manager's names and hire dates. Label the columns `Employee`, `Emp Hired`, `Manager`, and `Mgr Hired`, respectively.

```
SELECT w.last_name, w.hire_date, m.last_name, m.hire_date
FROM   employees w, employees m
WHERE  w.manager_id = m.employee_id
AND    w.hire_date <  m.hire_date;
-- OR
SELECT w.last_name, w.hire_date, m.last_name, m.hire_date
FROM   employees w JOIN employees m
ON    (w.manager_id = m.employee_id)
WHERE    w.hire_date <  m.hire_date;
```

Practice 5 Solutions

Determine the validity of the following three statements. Circle either True or False.

1. Group functions work across many rows to produce one result.

   True

2. Group functions include nulls in calculations.

   False. Group functions ignore null values. If you want to include null values, use the NVL function.

3. The `WHERE` clause restricts rows prior to inclusion in a group calculation.

   True

4. Display the highest, lowest, sum, and average salary of all employees. Label the columns `Maximum`, `Minimum`, `Sum`, and `Average`, respectively. Round your results to the nearest whole number. Place your SQL statement in a text file named `lab5_6.sql`.

```
SELECT   ROUND(MAX(salary),0) "Maximum",
         ROUND(MIN(salary),0) "Minimum",
         ROUND(SUM(salary),0) "Sum",
         ROUND(AVG(salary),0) "Average"
FROM     employees;
```

5. Modify the query in `lab5_4.sql` to display the minimum, maximum, sum, and average salary for each job type. Resave `lab5_4.sql` to `lab5_5.sql`. Run the statement in `lab5_5.sql`.

```
SELECT   job_id, ROUND(MAX(salary),0) "Maximum",
                 ROUND(MIN(salary),0) "Minimum",
                 ROUND(SUM(salary),0) "Sum",
                 ROUND(AVG(salary),0) "Average"
FROM     employees
GROUP BY job_id;
```

Practice 5 Solutions (continued)

6. Write a query to display the number of people with the same job.

```
SELECT   job_id, COUNT(*)
FROM     employees
GROUP BY job_id;
```

7. Determine the number of managers without listing them. Label the column `Number of Managers`. **Hint**: Use the `MANAGER_ID` column to determine the number of managers.

```
SELECT   COUNT(DISTINCT manager_id) "Number of Managers"
FROM     employees;
```

8. Write a query that displays the difference between the highest and lowest salaries. Label the column `DIFFERENCE`.

```
SELECT   MAX(salary) - MIN(salary) DIFFERENCE
FROM     employees;
```

If you have time, complete the following exercises.

9. Display the manager number and the salary of the lowest paid employee for that manager. Exclude anyone whose manager is not known. Exclude any groups where the minimum salary is less than $6,000. Sort the output in descending order of salary.

```
SELECT   manager_id, MIN(salary)
FROM     employees
WHERE    manager_id IS NOT NULL
GROUP BY manager_id
HAVING   MIN(salary) > 6000
ORDER BY MIN(salary) DESC;
```

10. Write a query to display each department's name, location, number of employees, and the average salary for all employees in that department. Label the columns `Name`, `Location`, `Number of People`, and `Salary`, respectively. Round the average salary to two decimal places.

```
SELECT   d.department_name "Name", d.location_id "Location",
         COUNT(*) "Number of People",
         ROUND(AVG(salary),2) "Salary"
FROM     employees e, departments d
WHERE    e.department_id = d.department_id
GROUP BY d.department_name, d.location_id;
```

Practice 5 Solutions (continued)

If you want an extra challenge, complete the following exercises:

11. Create a query that will display the total number of employees and, of that total, the number of employees hired in 1995, 1996, 1997, and 1998. Create appropriate column headings.

```
SELECT  COUNT(*) total,
        SUM(DECODE(TO_CHAR(hire_date, 'YYYY'),1995,1,0))"1995",
        SUM(DECODE(TO_CHAR(hire_date, 'YYYY'),1996,1,0))"1996",
        SUM(DECODE(TO_CHAR(hire_date, 'YYYY'),1997,1,0))"1997",
        SUM(DECODE(TO_CHAR(hire_date, 'YYYY'),1998,1,0))"1998"
FROM    employees;
```

12. Create a matrix query to display the job, the salary for that job based on department number, and the total salary for that job, for departments 20, 50, 80, and 90, giving each column an appropriate heading.

```
SELECT   job_id "Job",
         SUM(DECODE(department_id , 20, salary)) "Dept 20",
         SUM(DECODE(department_id , 50, salary)) "Dept 50",
         SUM(DECODE(department_id , 80, salary)) "Dept 80",
         SUM(DECODE(department_id , 90, salary)) "Dept 90",
         SUM(salary) "Total"
FROM     employees
GROUP BY job_id;
```

Practice 6 Solutions

1. Write a query to display the last name and hire date of any employee in the same department as Zlotkey. Exclude Zlotkey.

```
SELECT last_name, hire_date
FROM   employees
WHERE  department_id = (SELECT department_id
                        FROM   employees
                        WHERE  last_name = 'Zlotkey')
AND    last_nae <> 'Zlotkey';
```

2. Create a query to display the employee numbers and last names of all employees who earn more than the average salary. Sort the results in descending order of salary.

```
SELECT employee_id, last_name
FROM   employees
WHERE  salary > (SELECT AVG(salary)
                 FROM   employees);
```

3. Write a query that displays the employee numbers and last names of all employees who work in a department with any employee whose last name contains a *u*. Place your SQL statement in a text file named `lab6_3.sql`. Run your query.

```
SELECT employee_id, last_name
FROM   employees
WHERE  department_id IN (SELECT department_id
                         FROM   employees
                         WHERE  last_name like '%u%');
```

4. Display the last name, department number, and job ID of all employees whose department location ID is 1700.

```
SELECT last_name, department_id, job_id
FROM   employees
WHERE  department_id IN (SELECT department_id
                         FROM   departments
                         WHERE  location_id = 1700);
```

Practice 6 Solutions (continued)

5. Display the last name and salary of every employee who reports to King.

```
SELECT last_name, salary
FROM   employees
WHERE  manager_id = (SELECT employee_id
                     FROM   employees
                     WHERE  last_name = 'King');
```

6. Display the department number, last name, and job ID for every employee in the Executive department.

```
SELECT department_id, last_name, job_id
FROM   employees
WHERE  department_id IN (SELECT department_id
                         FROM   departments
                         WHERE  department_name = 'Executive');
```

If you have time, complete the following exercises:

7. Modify the query in `lab6_3.sql` to display the employee numbers, last names, and salaries of all employees who earn more than the average salary and who work in a department with any employee with a *u* in their name. Resave `lab6_3.sql` to `lab6_7.sql`. Run the statement in `lab6_7.sql`.

```
SELECT employee_id, last_name, salary
FROM   employees
WHERE  department_id IN (SELECT department_id
                         FROM   employees
                         WHERE  last_name like '%u%')
AND    salary > (SELECT AVG(salary)
                 FROM   employees);
```